UNIVERSIDAD
NACIONAL
DE COLOMBIA

Laboratory 1

Architectural Design

Jeisson Andrés Vergara Vargas

Software Architecture

2026-I

---

1. Objective

The objective of this lab is to design, build, deploy an test a simple software system using the monolithic architectural style, in order to have a first practical approach to the concepts of **structures** and **properties** of the system.

2. Prerequisites

A computer with a Unix-based OS and Docker installed.

3. (Mini-) Software Life Cycle

3.1. Design

Architectural Design Decisions:

- A sofwtare system with a monolitic architecture (**architectural style**).
- Two components: a database and a monolith.
- A database using MySQL (DBMS).
- A monolith using Python (programming language) and Flask (framework).
- A layered architecture into the monolith: *templates* > *controllers* > *services* > *respositories* > *models* (**architectural pattern**).

3.2. Construction

Create a **swarch** folder.

3.2.1. Model

a. Into the **swarch** folder, create a **models** folder.

b. Into the **models** folder, create a **grade.py** file:

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class Grade(db.Model):
    __tablename__ = 'grades'
    id = db.Column(db.Integer, primary_key=True)
    student_name = db.Column(db.String(100), nullable=False)
    subject = db.Column(db.String(100), nullable=False)
    score = db.Column(db.Float, nullable=False)
```

3.2.2. Repository

a. Into the **swarch** folder, create a **respositories** folder.

b. Into the **respositories** folder, create a **grade_repository.py** file:

```
from models.grade import Grade, db

def get_all():
    return Grade.query.all()

def add(student_name, subject, score):
    new_grade = Grade(student_name=student_name, subject=subject,
score=score)
    db.session.add(new_grade)
    db.session.commit()

def delete_by_id(grade_id):
    grade = Grade.query.get(grade_id)
    if grade:
        db.session.delete(grade)
        db.session.commit()
```

3.2.3. Service

a. Into the **swarch** folder, create a **services** folder.

b. Into the **services** folder, create a **grade_service.py** file:

```
from repositories import grade_repository

def list_grades():
    return grade_repository.get_all()

def create_grade(student_name, subject, score):
    grade_repository.add(student_name, subject, score)

def delete_grade(grade_id):
    grade_repository.delete_by_id(grade_id)
```

3.2.4. Controller

a. Into the **swarch** folder, create a **controllers** folder.

b. Into the **controllers** folder, create a **grade_controller.py** file:

```
from flask import Blueprint, render_template, request, redirect, url_for
from services import grade_service

grade_bp = Blueprint('grade_bp', __name__)

@grade_bp.route('/')
def index():
    grades = grade_service.list_grades()
    return render_template('grade_list.html', grades=grades)

@grade_bp.route('/add', methods=['POST'])
def add():
    student = request.form['student_name']
    subject = request.form['subject']
    score = float(request.form['score'])
    grade_service.create_grade(student, subject, score)
    return redirect(url_for('grade_bp.index'))

@grade_bp.route('/delete/<int:grade_id>')
def delete(grade_id):
    grade_service.delete_grade(grade_id)
    return redirect(url_for('grade_bp.index'))
```

3.2.5. Template

a. Into the **swarch** folder, create a **templates** folder.

b. Into the **templates** folder, create a **base.html** file:

```
<!DOCTYPE html>
<html>
```

```
<head>
    <title>SwArch2025ii</title>
</head>
<body>
    <h1>Gestor de Calificaciones</h1>
    {% block content %}{% endblock %}
</body>
</html>
```

c. Into the **templates** folder, create a **grade_list.html** file:

```
{% extends "base.html" %}

{% block content %}
<form action="/add" method="post">
    <input type="text" name="student_name" placeholder="Nombre del
estudiante" required>
    <input type="text" name="subject" placeholder="Asignatura" required>
    <input type="number" step="0.1" name="score"
placeholder="Calificación" required>
    <input type="submit" value="Agregar">
</form>
<ul>
    {% for grade in grades %}
        <li>
            {{ grade.student_name }} - {{ grade.subject }}: {{ grade.score
}}
            <a href="/delete/{{ grade.id }}">Eliminar</a>
        </li>
    {% endfor %}
</ul>
{% endblock %}
```

3.2.6. Configuration Files

a. Into the **swarch** folder, create a **config.py** file:

```
import os

DB_CONFIG = {
    'SQLALCHEMY_DATABASE_URI': os.getenv('DATABASE_URL'),
    'SQLALCHEMY_TRACK_MODIFICATIONS': False
}
```

b. Into the **swarch** folder, create a **requirements.txt** file:

```
Flask==2.3.3
Flask-SQLAlchemy==3.1.1
mysqlclient==2.2.0
```

c. Into the **swarch** folder, create an **app.py** file:

```python
from flask import Flask
from config import DB_CONFIG
from models.grade import db
from controllers.grade_controller import grade_bp

app = Flask(__name__)
app.config.update(DB_CONFIG)
db.init_app(app)

app.register_blueprint(grade_bp)

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(host='0.0.0.0', port=5000, debug=True)
```

d. Into the **swarch** folder, create a **Dockerfile** file:

```dockerfile
FROM python:3.11-slim

# Instalamos dependencias necesarias para compilar mysqlclient
RUN apt-get update && apt-get install -y \
    gcc \
    default-libmysqlclient-dev \
    build-essential \
    pkg-config \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

e. Into the **swarch** folder, create a **docker-compose.yml** file:

```yaml
services:
  swarch-mo:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - swarch-db
    environment:
      - DATABASE_URL=mysql://root:123@swarch-db/swarch-db
    volumes:
      - .:/app

  swarch-db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: 123
      MYSQL_DATABASE: swarch-db
    ports:
      - "3306:3306"
```

3.3. Deployment

Create the Docker images and generate the Docker containers through the execution of the following command:

```
docker-compose up --build
```

3.4. Testing

a. Open the system in a web browser: `http://localhost:8080`.

b. Create a new element (student + subject + grade) using the user interface.

c. Check the new element in the database component:

```
docker exec -it swarch-db sh
```

```
mysql -u root -p
```

Password = *123*

```
SHOW DATABASES;
```

```
USE swarch-db;
```

```
SELECT * FROM grades;
```

4. Delivery

4.1. Deliverable

- Full name.
- Graphical representation of the system structure(s).
- Description of five (5) identified system properties